ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

BÁO CÁO

CÔNG NGHỆ MẠNG KHẢ LẬP TRÌNH

Đề tài: Dynamic Access Control

GVHD: TS. Phan Xuân Thiện

Nhóm sinh viên thực hiện:

Lương Minh Anh - 22520050

Nguyễn Gia Linh - 22520768

Nguyễn Khánh Linh - 22520769

Trần Thiên Thanh - 22521367

Phạm Thị Cẩm Tiên – 22521473

Năm học: 2025 - 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin trân trọng cảm ơn thầy Phan Xuân Thiện - giảng viên hướng dẫn đồ án môn học Công nghệ mạng khả lập trình, người đã trực tiếp chỉ bảo và hướng dẫn tận tình cho chúng em trong suốt quá trình thực hiện và hoàn thành đồ án.

Với tất cả sự cố gắng, nhóm chúng em đã hoàn thành đồ án theo đúng kế hoạch đã đề ra và hoàn thiện hết sức có thể. Tuy nhiên, do kinh nghiệm và kiến thức thực tiễn còn hạn chế, đồ án không tránh khỏi những thiếu sót. Vì vậy, chúng em rất mong nhận được những ý kiến đóng góp quý báu và sự chỉ dẫn thêm từ thầy để đề tài được hoàn thiện hơn. Và cũng xin cảm ơn các thành viên trong nhóm đã nỗ lực đóng góp, thực hiện công việc của mình một cách tốt nhất để hoàn thành đồ án.

Chúng em xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, tháng 12 năm 2025

Nhóm sinh viên thực hiện

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

MỤC LỤC

LỜI CẢM ƠN 2
NHẬN XÉT CỦA GIẢNG VIÊN 3
MỤC LỤC 4
MỤC LỤC HÌNH ẢNH 5
1. Tổng quan 6
1.1. Bối cảnh 6
1.2. Đặt vấn đề 7
1.3. Mục tiêu của đề tài 7
2. Kiến thức nền tảng 8
2.1. Mạng khả lập trình (Software-Defined Networking – SDN) và Openflow[1][2] ... 8
2.2. Dynamic Access Control (DAC – Kiểm soát truy cập động)[3][4][5] 8
2.3. Mô hình ABAC (Attribute-Based Access Control - Kiểm soát truy cập dựa trên thuộc tính) [3][6][7] 9
3. Công nghệ sử dụng 9
3.1. Ryu Controller [8][9][10] 9
3.2. Open vSwitch (OVS) [11][12][13][14] 10
3.3. Mininet [15] 10
4. Mô hình 11
5. Kịch bản triển khai 12
5.1. Kịch bản 1: Kiểm soát truy cập động dựa trên vai trò trong mạng SDN 12
5.2. Kịch bản 2: ABAC theo Vai trò và Thời gian 22
6. Ưu nhược điểm 30
6.1. Kịch bản 1 30
6.2. Kịch bản 2 31
7. Tổng kết 32
TÀI LIỆU THAM KHẢO 33

MỤC LỤC HÌNH ẢNH

Hình 4.1. Mô hình mạng triển khai 11
Hình 5.1. Cấu hình controller cho kịch bản 1 (1) 13
Hình 5.2. Cấu hình controller cho kịch bản 1 (2) 14
Hình 5.3. Cấu hình controller cho kịch bản 1 (3) 15
Hình 5.4. Cấu hình controller cho kịch bản 1 (4) 16
Hình 5.5. Cấu hình controller cho kịch bản 1 (5) 16
Hình 5.6. Cấu hình controller cho kịch bản 1 (6) 17
Hình 5.7. Khởi chạy topology và controller 17
Hình 5.8. Sales host ping đến IT host 18
Hình 5.9. Sales host ping đến AppServer 18
Hình 5.10. IT host ping đến AppServer 19
Hình 5.11. Visitor host ping đến Sales 20
Hình 5.12. Visitor host ping đến AppServer 20
Hình 5.13. Unknown host ping đến Sales host 21
Hình 5.14. Unknown host ping đến AppServer 21
Hình 5.15. Cấu hình controller cho kịch bản 2 (1) 23
Hình 5.16. Cấu hình controller cho kịch bản 2 (2) 23
Hình 5.17. Cấu hình controller cho kịch bản 2 (3) 23
Hình 5.18. Cấu hình controller cho kịch bản 2 (4) 24
Hình 5.19. Cấu hình controller cho kịch bản 2 (5) 24
Hình 5.20. Cấu hình controller cho kịch bản 2 (6) 25
Hình 5.21. Cấu hình controller cho kịch bản 2 (7) 25
Hình 5.22. Cấu hình topology (1) 26
Hình 5.23. Cấu hình topology (2) 26
Hình 5.24. Khởi chạy topology 27
Hình 5.25. Khởi chạy controller 27
Hình 5.26. Bật dịch vụ http_server và ssh trên app_server 27
Hình 5.27. Test ping nội bộ 27
Hình 5.28. Kết quả đọc nội dung tài liệu chung (trong giờ làm việc) 28
Hình 5.29. Kết quả đọc nội dung tài liệu chung (ngoài giờ làm việc) 28
Hình 5.30. Log phía controller (ngoài giờ làm việc) 29
Hình 5.31. Kết quả ssh đến app_server 29
Hình 5.32. Log phía controller (trong giờ làm việc) 30

1. Tổng quan

1.1. Bối cảnh

Trong thời đại kỹ thuật số và cách mạng công nghiệp 4.0, mạng máy tính và hạ tầng viễn thông đã trở thành xương sống của hầu hết các hoạt động kinh tế, giáo dục, nghiên cứu và đời sống xã hội. Các tổ chức hiện nay đang vận hành những mạng lưới phức tạp với hàng nghìn, thậm chí hàng triệu thiết bị kết nối, từ máy tính cá nhân, máy chủ, thiết bị di động đến các thiết bị IoT (Internet of Things). Việc mở rộng này dẫn đến mạng trở nên phân tán, động và đa dạng, với các lưu lượng dữ liệu liên tục biến đổi về khối lượng, loại hình và mức độ nhạy cảm. Đồng thời, các tổ chức phải đảm bảo khả năng truy cập an toàn cho nhiều đối tượng khác nhau, bao gồm nhân viên nội bộ, đối tác kinh doanh, khách hàng và các thiết bị bên ngoài, mỗi đối tượng có mức độ quyền truy cập khác nhau và yêu cầu khác nhau về bảo mật.

Cùng với sự phát triển này, các mối đe dọa mạng cũng ngày càng tinh vi và đa dạng, bao gồm xâm nhập trái phép, tấn công từ chối dịch vụ (DDoS), phần mềm độc hại, khai thác lỗ hổng, rò rỉ dữ liệu và các hành vi truy cập không phù hợp. Trong môi trường mạng truyền thống, việc giám sát, kiểm soát và phân quyền truy cập trở nên ngày càng khó khăn, đặc biệt khi mạng phải xử lý đồng thời nhiều loại thiết bị và nhiều nhóm người dùng với các quyền truy cập khác nhau. Hơn nữa, các yêu cầu về linh hoạt và khả năng mở rộng của mạng hiện đại cũng tăng cao: các tổ chức cần khả năng thay đổi cấu hình, định tuyến, phân tách tài nguyên, hoặc cập nhật chính sách bảo mật một cách nhanh chóng để phù hợp với nhu cầu kinh doanh, thay đổi của người dùng và điều kiện môi trường.

Các mô hình kiểm soát truy cập truyền thống như Discretionary Access Control (DAC) hay Role-Based Access Control (RBAC) bắt đầu bộc lộ những hạn chế đáng kể trong môi trường mạng hiện đại. DAC phụ thuộc hoàn toàn vào quyết định chủ quan của người sở hữu tài nguyên, dễ dẫn đến rủi ro cấp quyền không phù hợp hoặc rò rỉ dữ liệu, trong khi RBAC dựa trên vai trò cố định, thiếu khả năng thích ứng với các thay đổi động của người dùng, tài nguyên hoặc các điều kiện môi trường. Ngoài ra, các hệ thống mạng truyền thống cũng khó đáp ứng yêu cầu quản lý truy cập đồng thời trong các môi trường phân tán, đám mây hoặc các hệ thống ảo hóa, nơi số lượng người dùng, thiết bị và ứng dụng liên tục thay đổi.

Từ những thực trạng này, có thể thấy rằng quản lý và kiểm soát quyền truy cập trong các hệ thống mạng hiện đại đang là một thách thức cấp thiết. Các tổ chức cần một giải pháp mạng mới, vừa đảm bảo tính bảo mật, linh hoạt và khả năng mở rộng, vừa có thể thích ứng với các thay đổi liên tục về người dùng, thiết bị, ứng dụng và môi trường hoạt động, đồng thời giảm thiểu rủi ro do cấu hình thủ công hoặc quyền truy cập không phù hợp.

1.2. Đặt vấn đề

Trước những thách thức và yêu cầu phức tạp của mạng hiện đại, Software-Defined Networking (SDN) xuất hiện như một giải pháp mạng tiên tiến, cho phép tách biệt hoàn toàn tầng điều khiển khỏi tầng dữ liệu. Kiến trúc này cung cấp khả năng quản lý tập trung, lập trình mạng linh hoạt và khả năng điều phối lưu lượng dữ liệu theo chính sách một cách chủ động, khác hẳn với các mạng truyền thống. SDN mang lại cơ hội để triển khai các cơ chế kiểm soát truy cập hiệu quả, nhất quán và dễ dàng mở rộng, phù hợp với môi trường mạng phân tán, động và đa dạng.

Trong đó, Dynamic Access Control là cơ chế kiểm soát truy cập thích ứng, cho phép hệ thống xác định quyền truy cập dựa trên nhiều thuộc tính đồng thời, bao gồm thuộc tính người dùng, tài nguyên, thiết bị và điều kiện môi trường như thời gian, vị trí hoặc trạng thái mạng. Chính nhờ khả năng đánh giá theo thời gian thực, Dynamic Access Control có thể tự động điều chỉnh quyền truy cập, ngăn chặn truy cập trái phép và tối ưu hóa việc quản lý mạng mà không cần sự can thiệp thủ công.

Do đó, nghiên cứu và triển khai SDN kết hợp Dynamic Access Control là hướng đi tất yếu nhằm giải quyết các vấn đề về bảo mật, kiểm soát truy cập, tính linh hoạt và khả năng mở rộng trong các hệ thống mạng hiện đại. Giải pháp này không chỉ nâng cao mức độ an toàn và tin cậy của mạng, mà còn mở ra khả năng quản lý tập trung, tự động hóa và triển khai các chính sách bảo mật chi tiết, đáp ứng nhu cầu ngày càng đa dạng và phức tạp của các tổ chức trong bối cảnh CNTT hiện nay.

1.3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và triển khai một giải pháp Dynamic Access Control trong môi trường mạng khả lập trình (SDN), nhằm nâng cao tính bảo mật, linh hoạt và khả năng quản lý truy cập của hệ thống mạng hiện đại. Cụ thể, đề tài hướng tới:

- Phân tích và đánh giá các cơ chế kiểm soát truy cập hiện có trong các mô hình mạng truyền thống và SDN, làm rõ ưu điểm, hạn chế và nhu cầu cải tiến trong môi trường mạng phân tán, động và đa người dùng.
- Thiết kế mô hình Dynamic Access Control tích hợp với SDN, dựa trên thuộc tính người dùng, tài nguyên, trạng thái thiết bị và điều kiện môi trường, để tự động phân quyền truy cập theo thời gian thực.
- Triển khai thử nghiệm mô hình trên môi trường giả lập sử dụng các công cụ như Ryu Controller, Open vSwitch và Mininet, nhằm kiểm chứng tính khả thi, hiệu quả và mức độ bảo mật của giải pháp.
- Đánh giá hiệu quả của mô hình, bao gồm khả năng ngăn chặn truy cập trái phép, tính linh hoạt trong quản lý truy cập, khả năng mở rộng và tác động đến hiệu suất

mạng, từ đó rút ra các khuyến nghị triển khai thực tế trong doanh nghiệp và môi trường mạng ảo hóa.

2. Kiến thức nền tảng

2.1. Mạng khả lập trình (Software-Defined Networking – SDN) và Openflow[1][2]

Mạng khả lập trình (Software-Defined Networking – SDN) là một kiến trúc mạng tiên tiến, cho phép tách biệt hoàn toàn giữa tầng điều khiển (control plane) và tầng dữ liệu (data plane), từ đó mang lại khả năng lập trình và quản lý mạng linh hoạt hơn so với các kiến trúc truyền thống. Trong mô hình mạng SDN, các thiết bị mạng như switch và router trở thành các phần tử dữ liệu thu động, chịu sự điều khiển từ bộ điều khiển tập trung (SDN Controller). SDN Controller là nơi định nghĩa các chính sách, đường đi của lưu lượng mạng, cũng như các quy tắc quản lý mạng. Openflow là một giao thức tiêu chuẩn trong SDN, cung cập cơ chế để bộ điều khiển có thể trực tiếp lập trình các luồng dữ liệu (flows) trên switch, bao gòm các quyết định chuyển tiếp, lọc hoặc xử lý các gói tin đi vào đi ra trong mạng dựa trên các tiêu chí cụ thể như địa chỉ IP, cổng hoặc giao thức. Kiến trúc này không chỉ nâng cao khả năng kiểm soát mạng một cách tập trung, linh hoạt mà còn hỗ trợ các ứng dụngg bảo mật tiên tiến, tối ưu hóa hiệu xuất, cân bằng tải và khả năng tự động phản ứng trước các thay đổi trong mạng. Nhở khả năng lập trình và chính sách hóa luồng dữ liệu, SDN/Openflow trở thành nền tảng lý tưởng cho các giải pháp mạng hiện đại như kiểm soát truy cập động, mạng ảo hóa và quản lý tài nguyên mạng theo thời gian thực.

2.2. Dynamic Access Control (DAC – Kiểm soát truy cập động)[3][4][5]

Dynamic Access Control là một cơ chế quản lý truy cập tiên tiến, trong đó việc cấp quyền truy cập không dựa trên các chính sách tĩnh hay quyền gán thủ công mà được quyết định thông qua quá trình đánh giá động các thuộc tính và điều kiện ngữ cảnh tại thời điểm truy cập. Cơ chế này thường được xây dựng trên nền tảng của mô hình kiểm soát truy cập dựa trên thuộc tính (Attribute-Based Access Control – ABAC), cho phép hệ thống sử dụng nhiều loại thông tin như thuộc tính người dùng, thuộc tính tài, điều kiện môi trường và các đặc trưng hành vi để đưa ra các quyết định truy cập chính xác và thích ứng với từng điều kiện.

Không giống với các mô hình truyền thống, như Discretionary Access Control (nơi quyền truy cập phụ thuộc vào quyết định của chủ sở hữu tài nguyên) hay Mandatory Access Control (nơi chính sách được áp dụng cố định bởi hệ thống), Dynamic Access Control vận hành theo phương thức linh họa hơn. Quyền truy cập có thể được điều chỉnh theo thời gian thực khi bất kỳ thuộc tính liên quan nào thay đổi. Nhờ đó hệ thông duy trì được quyền kiểm soát lên tục và duy trì được độ an toàn cao nhưng vẫn đảm bảo được tính linh hoạt trong môi trường vận hành. Đây là một yếu tố vô cùng quan trọng và cần thiết trong bối cảnh hiện nay – nơi mà các hệ thông ngày càng phân tán, động và phức tạp – chẳng hạn như

điện toàn đám mây, các môi trường doanh nghiệp lớn hoặc kiến trúc mạng khả lập trình (SDN).

Dynamic Access Control cũng hỗ trợ thêm khả năng quản trị tập trung. Trong đó, mọi chính sách bảo mật được định nghĩa tại một điểm điều khiển trung tâm và tự động áp dụng cho các tài nguyên và người dùng trên toàn hệ thống. Các tiếp cận hày giúp giảm nguy cơ do cấu hình thủ công bị sai sót; đồng thời, duy trình tính nhất quán của chính sách an ninh. Bên cạnh đó, khả năng giám sát, kiểm tra và ghi nhật ký theo chính sách động cũng giúp việc phát hiện kịp thời các hành vi bất thường trong mạng, từ đó xử lý kịp thời trước các cuộc tấn công mạng. Chính vì những ưu điểm về khả năng thích ứng, tính linh hoạt và độ an toàn cao, Dynamic Access Control đang trở thành một trong những hướng tiếp cập quan trọng trong thiết kế các hệ thống quản lý truy cập hiện đại.

2.3. Mô hình ABAC (Attribute-Based Access Control - Kiểm soát truy cập dựa trên thuộc tính) [3][6][7]

Mô hình ABAC (Attribute-Based Access Control – Kiểm soát truy cập dựa trên thuộc tính) là một cơ chế kiểm soát truy cập tiên tiến, trong đó quyền truy cập tài nghuyên được xác định dựa trên một tập các thuộc tính liên quan đến người dùng, tài nguyên, hành vi và môi trường hoạt động, thay vì dựa trên danh sách quyền tĩnh hay vai trò cố định. Các thuộc tính này có thể bao gồm thông tin về vai trò hoặc cấp bậc của người dùng, loại và mức độ nhạy cảm của tài nguyên, điều kiện môi trường như thời gian, vị trí hoặc trạng thái hệ thống, cũng như các yếu tố hành vi khác. Các quyết định truy cập trong Attribute-Based Access Control được được ra thông qua việc đánh giá các chính sách truy cập (access policies) – được định nghĩa bằng các quy tắt logic kết hợp với các thuộc tính, giúp hệ thông tự động xác định quyền truy cập của một người dùng hoặc một nhóm người dùng một cách chính xác, nhất quán và linh hoạt. Mô hình Attribute-Based Access Control cho phép thích ứng với các môi trường mạng phức tạp, phân tán và thay đổi liên tuc. Đồng thời Attribute-Based Access Control còn hỗ trợ khả năng quản trị tập trung và giảm thiểu rủi ro do sai sót trong quá trình cấu hình thủ công. Nhờ đặc tính linh hoạt và khả năng mở rộng cao, Attribute-Based Access Control đang trở thành một trong những phương pháp kiểm soát truy cập ưu việt, đặc biệt phù hợp với các hệ thống mạng hiện đại như điện toán đám mây, mạng khả lập trình (SDN) và các môi trường doanh nghiệp lớn – nơi yêu cầu bảo mật động và kiểm soát truy cập tinh vi.

3. Công nghệ sử dụng

3.1. Ryu Controller [8][9][10]

Ryu Controller là một nền tảng điều khiển mạng mã nguồn mở nỗi bật trong lĩnh vực Software-Defined Networking (SDN), được thiết kế nhằm hỗ trợ cho việc phát triển và triển khai các ứng dụng điều khiển mạng một cách linh hoạt và hiệu quả. Được xây dựng dựa trên ngôn ngữ Python với kiến trúc hướng sự kiện (event-driven), Ryu cho phép các

nhà nghiên cứu và kỹ sư mạng dễ dàng mô hình hóa các chính sách điều khiển, xử lý các sự kiện mạng, tương tác với các thiết bị Openflow, đồng thời, triển khai các cơ chế tối ưu hóa lưu lượng theo thời gian thực. Ryu cung cấp một bộ API phong phú, trừu tượng hóa các hoạt động phức tạp của giao thức Openflow, giúp quá trình phát triển ứng dụng trở nên đơn giản, mạch lạc và có thể mở rộng.

Trong kiến trúc SDN, Ryu đóng vai trò trung tâm trong tầng điều khiển (control plane), chịu trách nhiệm thu thập thông tin trạng thái từ các switch, phân tích luồng dữ liệu, phát hiện các hành vi bất thường, ra quyết định điều khiển và cài đặt các quy tắc luồng (flow rules) xuống các thiết bị mạng. Thông qua cơ chế xử lý linh hoạt, Ryu hỗ trợ triển khai đa dạng các chứng năng mạng như cân bằng tải, tối ưu hóa định tuyến, giám sát lưu lượng, phân đoạn mạng, phát hiện tấn công mạng, hoặc kiểm soát truy cập động dựa trên chính sách. Nhờ đặc tính mã nguồn mở, tải liệu đầy đủ và cộng đồng hỗ trợ tích cực, Ryu hiện là công cụ quan trọng trong môi trường nghiên cứu và thử nghiệm SDN, cho phép mô phỏng và kiểm nghiệm các mô hình bảo mật, kiến trúc điều khiển và thuật toán quản lý mạng tiên tiến trước khi được đưa vào vận hành trong môi trường thực tế.

3.2. Open vSwitch (OVS) [11][12][13][14]

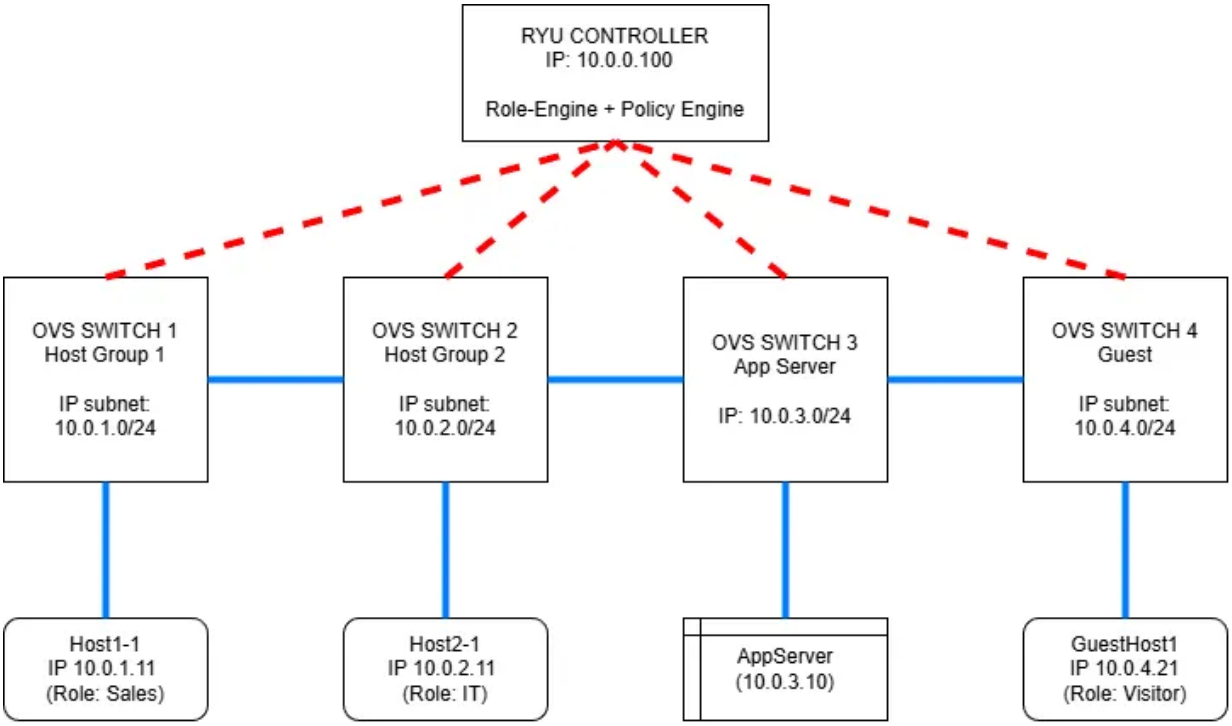
Open vSwitch (OVS) là một nền tảng chuyển mạch ảo mã nguồn mở được thiết kế chuyên biệt cho các môi trường ảo hóa và kiến trúc mạng hiện đại, đặc biệt là mạng khả lập trình (Software-Defined Networking – SDN). Open vSwitch cung cấp chức năng của một switch lớp 2/3 với khả năng mở rộng linh hoạt, hỗ trợ nhiều giao thức mạng quan trọng như Openflow, Netflow, sFlow, GRE, VXLAN và các cơ chế encapsulation khác, cho phép dễ dàng tích hợp vào các hạ tầng mạng ảo hoặc đám mây. Với kiến trúc theo mô hình kernel – user space, OVS tối ưu hiệu năng xử lý gói tin nhờ khả năng offload các tác vụ chuyển tiếp xuống nhân hệ điều hành trong khi vẫn duy trì sự linh hoạt của tầng điều khiển. Trong môi tường SDN, Open vSwitch đóng vai trò như là một switch dữ liệu có thể được lập trình trực tiếp bởi các bộ điều khiển như Ryu, ONOS hoặc OpenDaylight thông qua giao thức Openflow, cho phép triển khai các chính sách định tuyến, bảo mật và kiểm soát truy cập một các linh hoạt và tập trung. Nhờ tính ổn định, khả năng tương thích cao, hiệu suất tốt, OVS trở thành một thành phần chủ chốt trong nhiều hệ thống ảo hóa mạng, từ các nên tảng đám mây như OpenStack cho đến các phòng thí nghiệm nghiên cứu và triển khai SDN quy mô lớn.

3.3. Mininet [15]

Mininet là một công cụ mô phỏng mạng mã nguồn mở được sử dụng rộng rãi trong nghiên cứu và phát triển các giải pháp cho mạng khả lập trình (SDN), nhờ khả năng tái tạo một cách nhẹ nhàng nhưng chính xác hành vi của mạng thật. Thông qua việc tạo ra các thực thể mạng ảo như host, switch, router và liên kết ảo ngay trên một máy tính duy nhất, Mininet cho phép các nhà nghiên cứu và kỹ sư triển khai, kiểm thử và đánh giá các giao thức, chính sách hoặc ứng dụng mạng trong môi trường gần với thực tế mà không yêu cầu hạ tầng phần

cứng phức tạp. Công cụ còn hỗ trợ thêm các bộ điều khiển SDN như Ryu, ONOS và OpenDaylight; đồng thời tích hợp tốt với Open vSwitch để mô phỏng luồng dữ liệu và hành vi chuyển mạch theo giao thức Openflow. Nhờ đặc tính linh hoạt, dễ sử dụng và khả năng mở rộng tùy ý tùy theo các kịch bản, Mininet trở thành một nền tảng quan trọng trong đào tạo, thử nghiệm, và nghiên cứu học thuật liên quan đến mạng khả lập trình (SDN), giúp rút ngắn thời gian phát triển và nâng cao chất lượng cho các giải pháp mạng tiên tiến.

4. Mô hình



Hình 4.1. Mô hình mạng triển khai

Mô hình mạng được thiết kế gồm 4 miền mạng chính, mỗi miền đại diện cho một nhóm người dùng hoặc chức năng riêng biệt, được kết nối và điều phối thông quan kiến trúc SDN với bộ điều khiển Ryu làm trung tâm quản lý. Trong đó:

- Host Group 1 kết nối thông qua OVS SWITCH 1, sử dụng dải địa chỉ 10.0.1.0/24 được bố trí dành cho nhân viên Sales.
- Host Group 2 kết nối thông qua OVS SWITCH 2, sử dụng dải địa chỉ 10.0.2.0/24, phục vụ cho nhóm nhân viên IT.
- Hệ thống còn bao gồm 1 App Server được kết nối tại OVS SWITCH 3, với địa chỉ IP 10.0.3.10, đóng vai trò lưu trữ và cung cấp các ứng dụng hoặc tài nguyên quan trọng, cần áp dụng cơ chế kiểm soát truy cập.

- Ngoài ra, OVS SWITCH 4 đảm nhiệm việc kết nối với miền khách ngoài với dải địa chỉ 10.0.4.0/24, dành riêng cho các thiết bị không thuộc nội bộ doanh nghiệp.
- Toàn bộ hạn tầng được quản lý tập trung bởi Ryu Controller tại địa chỉ10.0.0.100, đây là nơi thực thi các chính sách điều khiển mạng và kiểm soát truy cập dựa trên quy tắc.
- Các nhóm người dùng nội bộ gồm Sales và IT được phép truy cập AppServer nhằm phục vụ nhu cầu cồng việc; trong khí đó, phân đoạn Guest bị từ chối truy cập nhằm đảm bảo tính bảo mật và cách ly tài nguyên quan trọng của doanh nghiệp khỏi những người dùng không đáng tin cậy.

⇨ Sự phân tách logic kết hợp với điều khiển tập trong của SDN giúp nâng cao tính linh hoạt, khả năng kiểm soát và mức độ an toàn cho tổng thể hệ thống mạng.

5. Kịch bản triển khai

5.1. Kịch bản 1: Kiểm soát truy cập động dựa trên vai trò trong mạng SDN

a. Mô tả

- Mục tiêu và phạm vi

Kịch bản triển khai hệ thống Dynamic Access Control trong môi trường Software-Defined Networking nhằm kiểm soát truy cập mạng một cách linh hoạt và tự động dựa trên vai trò của người dùng. Khác với các hệ thống Access Control List truyền thống được cấu hình tĩnh trên từng thiết bị, hệ thống Dynamic Access Control cho phép thay đổi chính sách truy cập tập trung tại controller và tự động cập nhật flow rules trên tất cả các switches trong thời gian thực. Mục tiêu chính là bảo vệ tài nguyên nhạy cảm AppServer khỏi truy cập trái phép, đồng thời cho phép các nhân viên thuộc phòng Sales và IT có thể truy cập hợp lệ. Hệ thống cũng cung cấp khả năng phát hiện và cảnh báo các thiết bị không xác định cố gắng truy cập vào mạng.

- Luồng hoạt động dynamic access control

Tính dynamic của hệ thống được thể hiện ở việc các quyết định điều khiển truy cập được thực hiện dựa trên nhu cầu thay vì được cấu hình trước cố định. Khi một host gửi packet lần đầu tiên, switch không có flow rule tương ứng sẽ thực hiện gửi Packet_In lên controller. Controller sau khi phân tích packet, xác định vai trò của host dựa trên địa chỉ IP, kiểm tra chính sách policy được thiết lập và đưa ra quyết định cho phép hoặc từ chối.

Nếu cho phép, controller sẽ tự động cài đặt flow rule xuống switch để các packets tiếp theo được xử lý trực tiếp. Nếu từ chối, controller sẽ cài đặt drop rule và log cảnh báo trong thời gian thực. Điểm quan trọng là khi chính sách policy được thay đổi hoặc cập nhật tại controller, các flow rules cũ sẽ tự động hết hạn sau idle_timeout và flow rules mới sẽ được

cài đặt dựa trên policy mới mà không cần cấu hình lại switches. Đây là ưu điểm lớn so với phương pháp kiểm soát truy cập tĩnh trên router hay firewall truyền thống, nơi mà mọi thay đổi đều yêu cầu cấu hình thủ công từng thiết bị.

b. Triển khai

b.1. Cơ chế gán vai trò động dựa trên Subnet

Thay vì gán vai trò cố định cho từng địa chỉ MAC trong chính sách, hệ thống sẽ sử dụng gán role động dựa trên subnet. Cơ chế này cho phép tự động xác định vai trò của bất kỳ thiết bị nào dựa trên địa chỉ IP mà không cần cấu hình trước.

```
ROLE_SUBNETS = {
    '10.0.1.0/24': 'Sales',      # Lớp mạng của Sales
    '10.0.2.0/24': 'IT',         # Lớp mạng của IT
    '10.0.4.0/24': 'Visitor'     # Lớp mạng của Visitor
}

APP_SERVER_SUBNET = '10.0.3.0/24'  # Lớp mạng của AppServer
ALLOWED_ROLES = ['Sales', 'IT']     # Các role được cho phép truy cập AppServer
```

Hình 5.1. Cấu hình controller cho kịch bản 1 (1)

Để xác định vai trò, chính sách chủ yếu dựa vào module ipaddress để thực hiện so sánh và kiểm tra subnet-matching dựa trên địa chỉ IP của host. Việc kiểm tra này có độ phức tạp O(1), giúp cho hệ thống xác định vai trò ngay lập tức mà không cần dựa vào database hay configuration file.

```python
def __init__(self, *args, **kwargs):
        super(RoleAccessControl, self).__init__(*args, **kwargs)
        self.mac_to_port = {}
        self.mac_to_ip = {}
        self.blocked_flows = set()
        self.role_networks = {ipaddress.ip_network(subnet): role
                              for subnet, role in ROLE_SUBNETS.items()}
        self.app_server_network = ipaddress.ip_network(APP_SERVER_SUBNET)


def _get_role_from_ip(self, ip_str):
    if not ip_str:
        return 'Unknown'

    try:
        ip_addr = ipaddress.ip_address(ip_str)

        # Thực hiện kiểm tra nếu IP thuộc subnet của AppServer
        if ip_addr in self.app_server_network:
            return 'AppServer'

        # Nếu không phải thuộc AppServer thì thực hiện kiểm tra dựa trên
        # dictionary gồm các subnet và role tương ứng để gán role cho IP
        for network, role in self.role_networks.items():
            if ip_addr in network:
                return role

        # Nếu không thuộc subnet đã định nghĩa thì gán role Unknown
        return 'Unknown'
    except ValueError:
        return 'Unknown'
```

Hình 5.2. Cấu hình controller cho kịch bản 1 (2)

Ưu điểm của phương pháp này là có tính mở rộng cao. Nếu hệ thống có thêm 100 nhân viên sales mới, chỉ cần DHCP Server cấp IP trong subnet 10.0.1.0/24 thì thiết bị của các nhân viên này sẽ tự động có role Sales. Vì vậy quản trị viên không cần phải cập nhật controller configuration hay khởi động lại dịch vụ. Bên cạnh đó, phương pháp này cũng phù hợp với kiến trúc mạng, vì các tổ chức thường phân chia subnet theo phòng ban như VLAN, do đó role sẽ được phân tự nhiên theo cấu trúc mạng hiện có. Cuối cùng là tính linh động, khi muốn thay đổi role của một thiết bị, quản trị viên chỉ cần đổi subnet thông qua VLAN assignment mà không cần phải thông qua controller.

b.2. Cơ chế đưa ra quyết định của policy

Policy Decision Point (PDP) là thành phần trọng yếu thực hiện đưa ra các quyết định truy cập động. Khác với cơ chế kiểm soát truy cập tĩnh là dựa vào rule được thiết lập sẵn, PDP sẽ đánh giá các flow-request ngay trong thời gian thực và đưa ra quyết định dựa trên chính sách policy được thiết lập hiện tại.

```python
def _check_access_policy(self, src_ip, dst_ip):
    role_src = self._get_role_from_ip(src_ip) # Kiểm tra role của source IP
    is_dst_appserver = self._is_app_server(dst_ip) # Kiểm tra nếu dest IP là AppServer

    # Trường hợp 1: User → AppServer
    if is_dst_appserver:
        if role_src == 'Unknown':
            return False, f"[!!!]  SECURITY ALERT: Unknown device ({src_ip}) attempted to access AppServer", role_src
        elif role_src in ALLOWED_ROLES:
            return True, f"Role '{role_src}' is allowed to access AppServer", role_src
        elif role_src == 'AppServer':
            # AppServer loopback - cho phép
            return True, "AppServer internal traffic", role_src
        else:
            return False, f"Role '{role_src}' is DENIED access to AppServer", role_src

    # Trường hợp 2: AppServer → User (luôn cho phép - response traffic)
    if role_src == 'AppServer':
        return True, "AppServer response traffic is allowed", role_src

    # Trường hợp 3: Unknown → bất kỳ đâu (cảnh báo nhưng có thể cho phép traffic thông thường)
    if role_src == 'Unknown':
        return True, f"[*!]  Unknown device ({src_ip}) in network - normal traffic allowed", role_src

    # Trường hợp 4: User ↔ User (luôn cho phép)
    return True, "Normal inter-user traffic is allowed", role_src
```

Hình 5.3. Cấu hình controller cho kịch bản 1 (3)

Trong đó cơ chế đưa ra quyết định dựa vào các trường hợp sau:

- Xử lý truy cập đến AppServer: Nếu role của source IP là Unknow thì system sẽ ngay lập tức từ chối và in ra log cảnh báo vì đây có thể là hành động reconnaissance của kẻ tấn công. Trong trường role source thuộc list role được cho phép thì việc truy cập sẽ được cho phép.
- AppServer muốn phản hồi traffic từ user: Đây là cần thiết để cho các giao thức dạng request-response hay TCP Connection hoạt động.
- Các trường hợp của user giao tiếp với nhau: Được cho phép tuy nhiên đối với trường hợp của IP mang role Unknown sẽ được log warning để quản trị viên có thể thông tin hoặc điều tra nếu cần thiết.

b.3. Cơ chế cài đặt flow rule

Sau khi PDP đã ra quyết định, controller sẽ thực hiện cài đặt flow rule vào switch. Đây chính là điểm tốt hơn của dynamic khi rule được tạo dự trên nhu cầu và có thể được tự động loại bỏ khi không còn sử dụng.

```
if allowed:
    actions = [parser.OFPActionOutput(out_port)]

    # Xác định priority dựa trên loại traffic
    if self._is_app_server(dst_ip):
        priority = 100  # Ưu tiên cao nhất cho traffic truy cập đến AppServer
        idle_timeout = 60
    elif role_src == 'AppServer':
        priority = 50
        idle_timeout = 60
    else:
        priority = 10
        idle_timeout = 30

    # Cài đặt flow rule
    match = parser.OFPMatch(eth_type=0x0800,
                            ipv4_src=src_ip,
                            ipv4_dst=dst_ip)
    self._add_flow(datapath, priority, match, actions,
                   idle_timeout=idle_timeout)
```

Hình 5.4. Cấu hình controller cho kịch bản 1 (4)

b.4. Quản lý flow rule time out

Chính sách cũng sử dụng Idle timeout để cho phép flow rule tự động expire. Khi không còn traffic nào nữa thì flow rule sẽ được loại bỏ sau 60 giây. Sau đó lại có traffic khi policy đã thay đổi thì flow mới sẽ bị từ chối nếu policy mới không cho phép truy cập. Đây chính là ưu điểm của Dynamic Access Control.

```
def _add_flow(self, datapath, priority, match, actions,
              idle_timeout=0, hard_timeout=0):
    inst = [parser.OFPInstructionActions(ofproto_v1_3.OFPIT_APPLY_ACTIONS,
                                         actions)]
    mod = parser.OFPFlowMod(datapath=datapath,
                            priority=priority,
                            match=match,
                            instructions=inst,
                            idle_timeout=idle_timeout,
                            hard_timeout=hard_timeout)
    datapath.send_msg(mod)
```

Hình 5.5. Cấu hình controller cho kịch bản 1 (5)

b.5. Xử lý sự kiện Packet-In

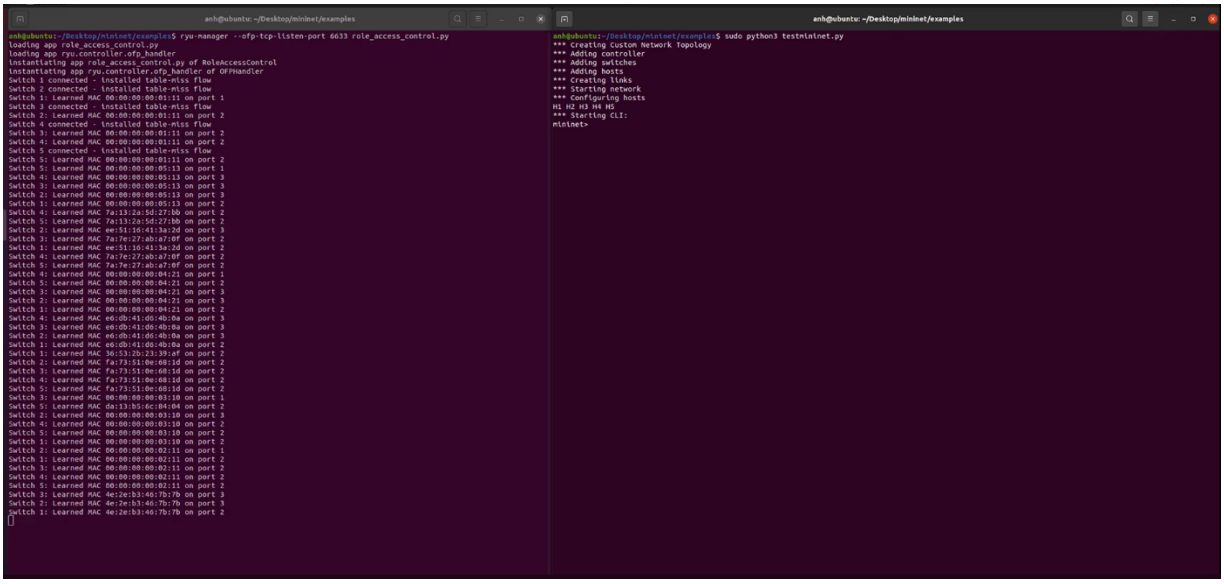
Mỗi khi một Packet-In gửi đến sẽ kích hoạt một vòng logic: trích xuất IP à Xác định vai trò à Kiểm tra chính sách à cài đặt rule.

```python
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg
    pkt = packet.Packet(data=msg.data)
    eth = pkt.get_protocol(ethernet.ethernet)
    src_ip, dst_ip = self._extract_ip_from_packet(pkt)

    role_src = self._get_role_from_ip(src_ip)

    allowed, reason, role_src = self._check_access_policy(src_ip, dst_ip)

    if allowed:
        self._add_flow(datapath, priority, match, actions, idle_timeout)

    else:
        self._add_drop_flow(datapath, priority, match, idle_timeout)
```

Hình 5.6. Cấu hình controller cho kịch bản 1 (6)

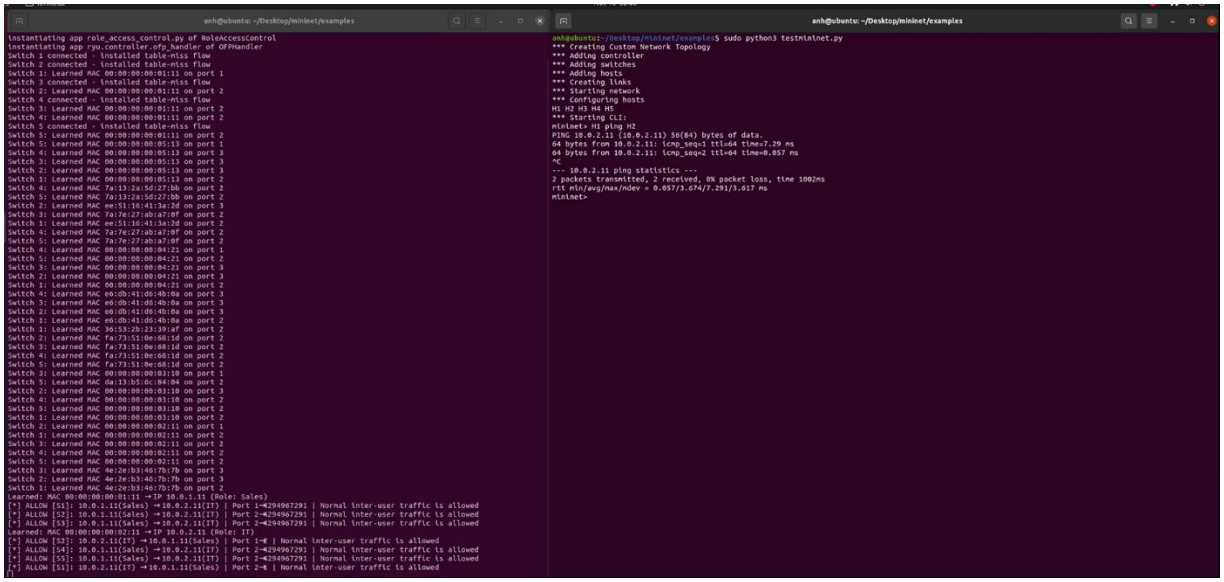
c. Demo

- Đầu tiên khởi chạy Ryu Controller với tập chính sách đã viết và mô hình mạng:



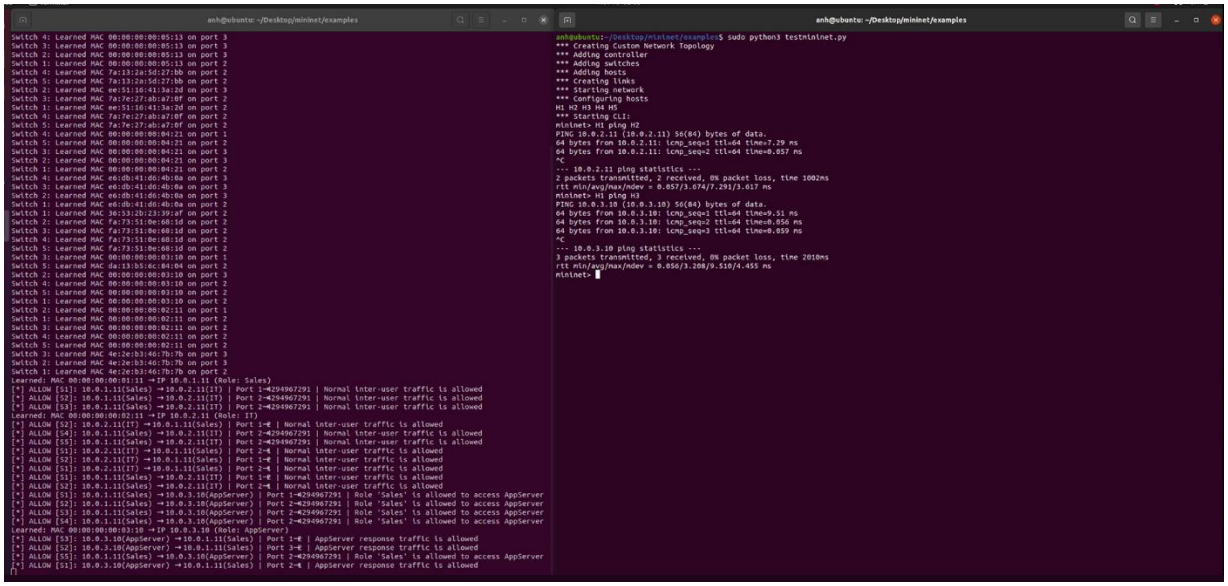
Hình 5.7. Khởi chạy topology và controller

- Thực hiện kết nối từ một host trong nhóm phòng Sales đến một host trong nhóm phòng IT. Có thể quan sát được hai host có thể kết nối được với nhau do theo chính sách thì nhân viên phòng Sales và phòng IT có thể kết nối lẫn nhau. Trong log được Ryu ghi lại cho thấy khi mới bắt đầu, controller chưa biết thông tin gì của các host. Khi có gói tin đầu tiên được gửi lên, controller học được ánh xạ giữa địa chỉ MAC và địa chỉ IP của host. Sau đó controller dựa vào địa chỉ IP của host mà xác định vai trò của người dùng trong mạng theo subnet đã được cấu hình trước đó. Từ đó mà controller có thể quyết định chính sách RBAC tương ứng và sinh flow rule cho phép hoặc từ chối.



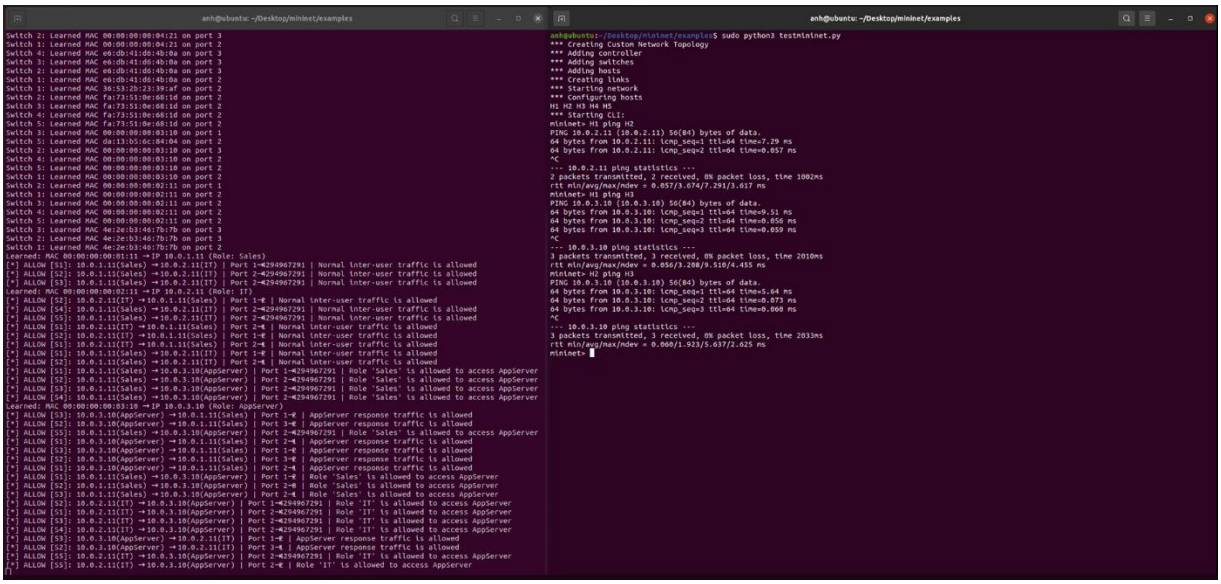
Hình 5.8. Sales host ping đến IT host

- Thực hiện kết nối giữa một host của phòng Sales đến AppServer. Theo chính sách thì các nhân viên của phòng Sales được phép truy cập đến AppServer nên khi thực hiện kết nối hai bên kết nối bình thường. Trong log của Ryu có thể thấy khi có một host mới (ở đây là AppServer), controller lại học ánh xạ giữa địa chỉ MAC và địa chỉ IP của host đó. Từ đó, controller mới dựa vào địa chỉ IP của hai host mà xác định chính sách RBAC tương ứng và sinh flow rule thích hợp.



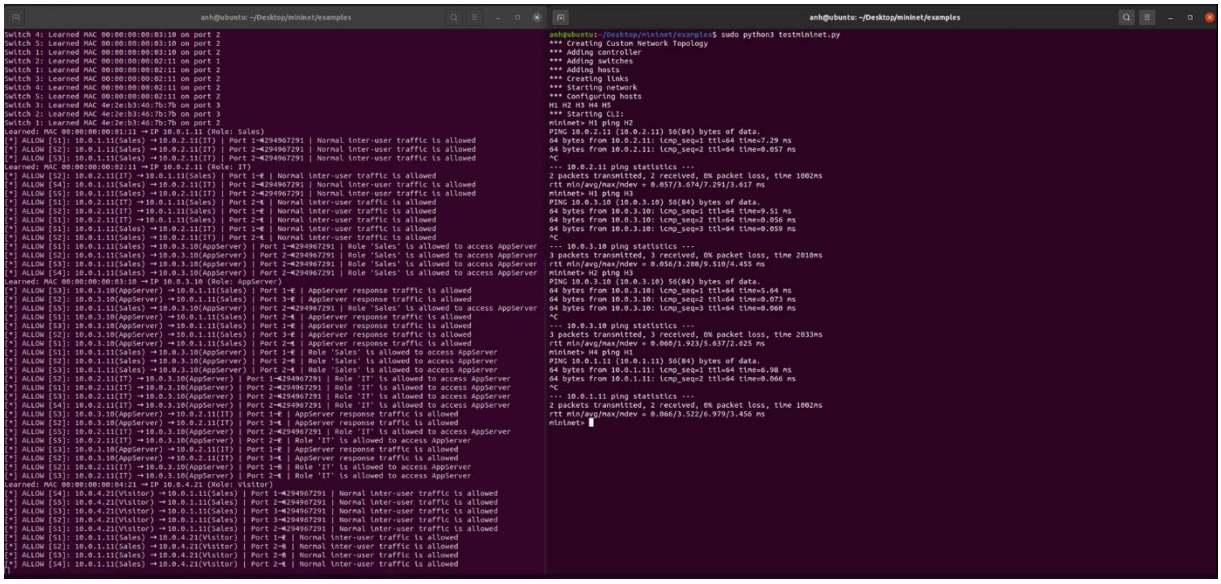
Hình 5.9. Sales host ping đến AppServer

- Ở testcase thứ ba, thực hiện kết nối giữa host của phòng IT và AppServer. Kết quả trả về tương tự như trường hợp trên. Trong log của Ryu ta có thể thấy được vì host này đã từng thực hiện kết nối nên controller đã học được ánh xạ giữa địa chỉ MAC và địa chỉ IP của host nên không cần học nữa mà chỉ cần xét địa chỉ IP để xác định chính sách để sinh flow rule cho phù hợp.



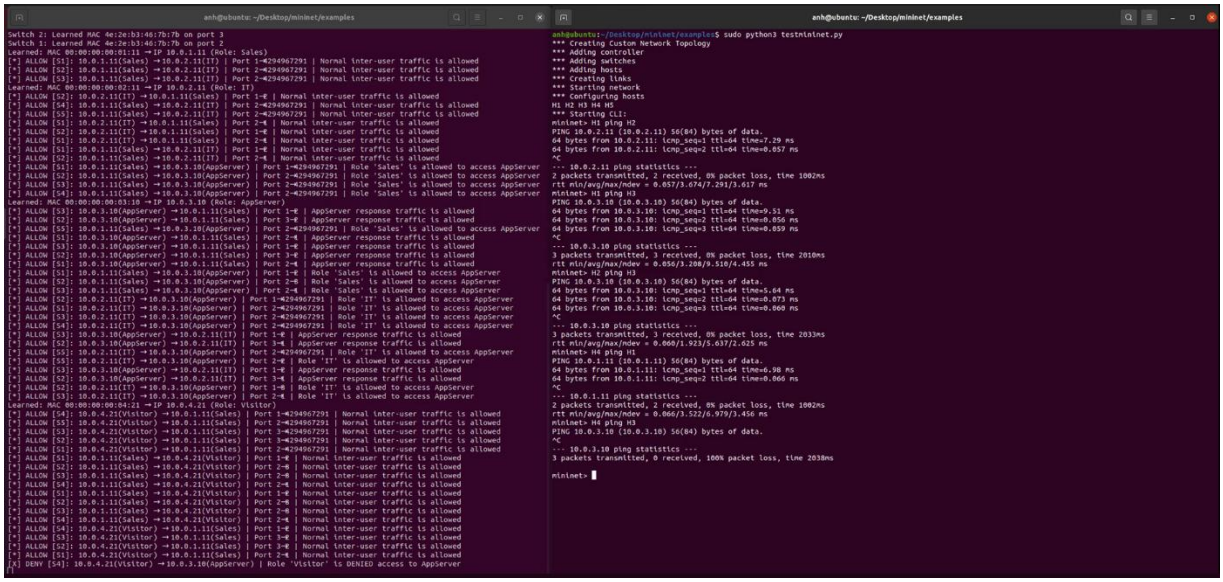
Hình 5.10. IT host ping đến AppServer

- Thực hiện kết nối giữa một host trong nhóm mạng của Guest với phòng Sales. Theo chính sách quy định các host của nhóm mạng Guest không bị cấm kết nối với nhóm mạng Sales và IT nên có thể thấy kết nối được thực hiện bình thường. Trong log của Ryu ta thấy controller lại học được một ánh xạ giữa địa chỉ MAC và địa chỉ IP mới của Visitor, kèm với đó là log allow (cho phép truy cập).



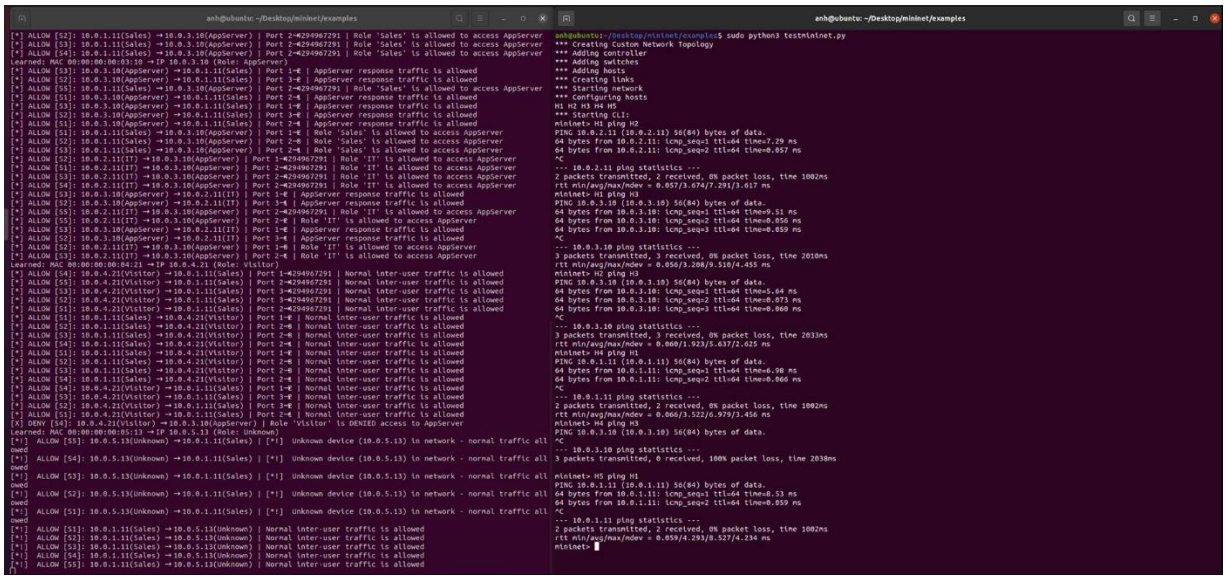
Hình 5.11. Visitor host ping đến Sales

- Tuy nhiên, trong chính sách quy định, không cho nhóm mạng Guest kết nối với AppServer. Vậy nên, khi thử nghiệm trường hợp kết nối giữa Visitor thuộc mạng Guest và AppServer ta thấy log của Ryu ghi lại thông báo Deny và cũng không có gói tin nào được gửi thành công.



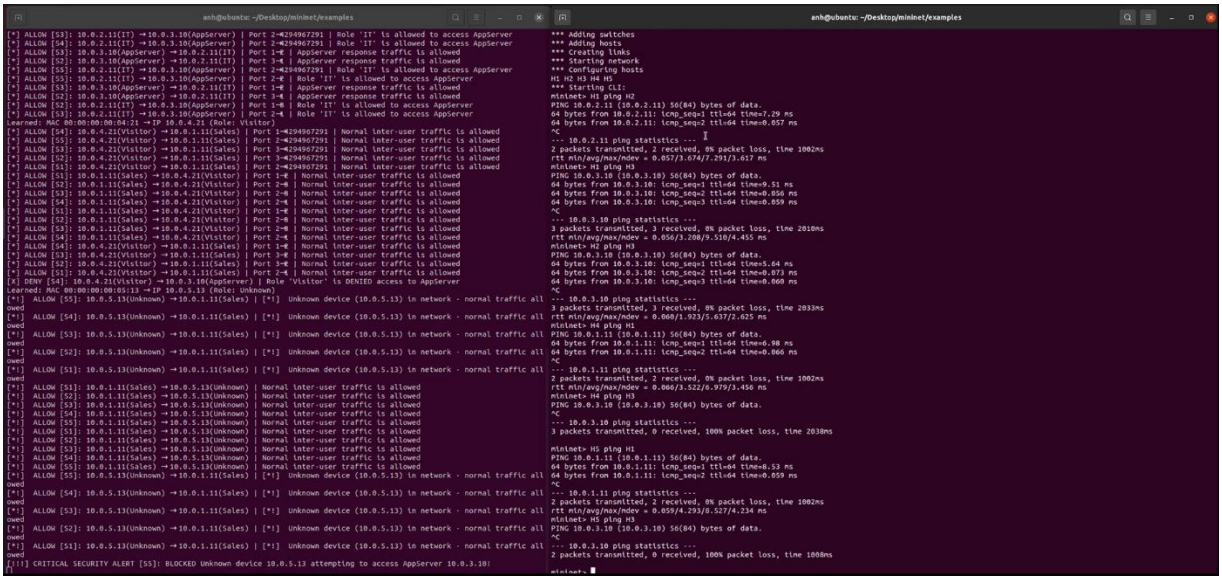
Hình 5.12. Visitor host ping đến AppServer

- Trường hợp tiếp theo là một Unknown host kết nối đến một host của phòng Sale. Trong chính sách không quy định lệnh cấm đối với các Unknown host đến các host phòng Sales và IT. Vậy nên việc kết nối vẫn diễn ra bình thường.



Hình 5.13. Unknown host ping đến Sales host

- Trường hợp cuối cùng, nhóm cho unknown host thực hiện kết nối với AppServer. Vì AppServer là tài nguyên quan trọng của và chỉ những người thuộc nội bộ doanh nghiệp mới được phép truy cập. Vậy nên ngay khi có kết nối lạ gửi đến AppServer, controller ngay lập tức cảnh báo và chặn truy cập để ngăn chặn hành vi nguy hiểm xảy ra.



Hình 5.14. Unknown host ping đến AppServer

c. Demo

Link video: https://www.youtube.com/watch?v=KsXDkk4X65A

- Các test case trong demo:
  - Test 1: Sales host ping đến IT host
  - Test 2: Sales host ping đến AppServer
  - Test 3: IT host ping đến AppServer
  - Test 4: Visitor host ping đến Sales
  - Test 5: Visitor host ping đến AppServer
  - Test 6: Unknown host ping đến Sales host
  - Test 7: Unknown host ping đến AppServer

5.2. Kịch bản 2: ABAC theo Vai trò và Thời gian

a. Mô tả

- Mục tiêu và phạm vi

Kịch bản đề xuất, triển khai một hệ thống kiểm soát truy cập động dựa trên thuộc tính ABAC (Attribute-Based Access Control), sử dụng thông tin về vai trò người dùng (Role) và thời gian truy cập (Time), nhằm xác định quyền hạn của người dùng đối với các tài nguyên. Mục tiêu là nâng cao tính bảo mật và tự động hóa trong quản lý và kiểm soát truy cập tài nguyên công ty linh hoạt và an toàn trong môi trường doanh nghiệp, chỉ cho phép nhân viên truy cập khi cần thiết, đồng thời hệ thống sẽ cảnh báo các hành vi cố gắng truy cập trái phép.

- Luồng hoạt động

Mô hình kiểm soát truy cập dựa trên thuộc tính, dựa trên các chính sách bao gồm tập hợp các thuộc tính của người dùng, tài nguyên và hành động để xem xét yêu cầu truy cập có được cấp phép không. Khi truy cập lần đầu, switch sẽ gửi Packet_In đến controller. Sau đó controller sẽ tiến hành thu thập các thuộc tính: role của người dùng (dựa trên định danh IP/MAC), Current Time, Resource và Action mong muốn, từ đó truy vấn đến chính sách để đưa quyết định cho phép hay từ chối.

Nếu được cho phép, controller tự động cài flow rules xuống switch để tiếp tục xử lí. Nếu không, cài drop rule và cảnh báo cho quản trị viên.

b. Triển khai

b.1. Cơ chế gán vai trò động dựa trên Subnet

Tương tự kịch bản 1, nhóm sẽ dựa vào địa chỉ IP, xét subnet để gán role tự động cho thiết bị gửi yêu cầu.

```python
# Config
ROLE_SUBNETS = {
    '10.0.1.0/24': 'Sales',
    '10.0.2.0/24': 'IT',
    '10.0.4.0/24': 'Visitor'
}

APP_SERVER_SUBNET = '10.0.3.0/24'
```

Hình 5.15. Cấu hình controller cho kịch bản 2 (1)

Bên cạnh vai trò, hệ thống sẽ lấy thông tin thời gian hiện tại để đối chiếu với thời gian bắt đầu và thời gian kết thúc của giờ làm việc để xét yêu cầu có được chấp nhận hay không. Nhằm tăng độ thực tế cho kịch bản, ở app_server (có subnet 10.0.3.0/24) nhóm có triển khai hai dịch vụ là http_server (port 80) để thiết bị có thể gửi yêu cầu, đọc được dữ liệu chung được lưu tại server này và dịch vụ ssh (port 22) được sử dụng cho các thiết bị có quyền truy cập và điều khiển từ xa (trong kịch bản này là role IT, subnet 10.0.2.0/24) để có thể tạo, thêm, xóa, sửa và thực thi các file dùng chung trên hệ thống.

```python
WORK_START = time(8, 0)
WORK_END   = time(16, 30)

READ_PORTS  = [80]  # port dùng cho read file
WRITE_PORTS = [22]  # port dùng cho write file
```

Hình 5.16. Cấu hình controller cho kịch bản 2 (2)

Để xác định vai trò của thiết bị, nhóm triển khai một hàm lấy vai trò, dựa vào module ipaddress kiểm tra subnet-matching tương tự kịch bản 1. Tùy vào địa chỉ IP, controller sẽ tự động gán vai trò tương ứng cho thiết bị, trong trường hợp có lỗi hoặc thiết bị thuộc subnet không được định nghĩa từ trước, mặc định vai trò sẽ là "Unknown".

```python
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    self.mac_to_port = {}
    self.role_networks = {ipaddress.ip_network(k): v for k, v in ROLE_SUBNETS.items()}
    self.app_net = ipaddress.ip_network(APP_SERVER_SUBNET)

def _get_role(self, ip):
    try:
        ipaddr = ipaddress.ip_address(ip)
        if ipaddr in self.app_net:
            return 'AppServer'
        for net, role in self.role_networks.items():
            if ipaddr in net:
                return role
        return 'Unknown'
    except:
        return 'Unknown'
```

Hình 5.17. Cấu hình controller cho kịch bản 2 (3)

Hàm xác định thời gian làm việc được triển khai dựa trên việc so sánh thời gian hiện tại được lấy từ hệ thống và thời gian làm việc được định nghĩa từ trước.

```
def _in_work_time(self):
    now = datetime.now().time()
    return WORK_START <= now <= WORK_END
```

Hình 5.18. Cấu hình controller cho kịch bản 2 (4)

Kịch bản 2 vẫn giữ được ưu điểm có tính mở rộng cao, bên cạnh đó vì xét trên nhiều yếu tố (vai trò, thời gian, loại tài nguyên) giúp kịch bản có độ linh hoạt và thực tế cao hơn, tuy nhiên nếu mô hình triển khai có quá nhiều yếu tố, có thể khiến phương pháp này trở nên phức tạp, giảm hiệu quả và khó bảo trì.

b.2. Cơ chế đưa ra quyết định của policy

Từ các thông tin thiết bị đã thu thập ở trên, hệ thống sẽ sử dụng cơ chế PDP để quyết định các yêu cầu có được thông qua hay không dựa trên các policy được cài đặt như sau:

```
def check_policy(self, src_ip, dst_ip, dst_port):
    role = self._get_role(src_ip)

    if ipaddress.ip_address(dst_ip) in self.app_net:
        if role == 'Visitor':
            return False, "Visitor denied access to AppServer"

        if role == 'Sales':
            if not self._in_work_time():
                return False, "Sales outside work time"
            if dst_port in READ_PORTS:
                return True, "Sales READ secret.txt"
            return False, "Sales WRITE/EXEC denied"

        if role == 'IT':
            if dst_port in READ_PORTS + WRITE_PORTS:
                return True, "IT full access"
            return False, "IT denied non-file port"

    return True, "Internal allow"
```

Hình 5.19. Cấu hình controller cho kịch bản 2 (5)

Cơ chế được triển khai như sau:

- Role = “Visitor”, không chấp nhận bất kỳ loại yêu cầu nào đến app_server (ở đây bao gồm các yêu cầu có thể thực hiện qua cổng 80 và 22), có thể chấp nhận ping trong mạng nội bộ.
- Role = “Sales”, chỉ cho phép truy cập tới port 80 để yêu cầu đọc tài liệu trong giờ làm việc, ngoài giờ làm việc, yêu cầu không được chấp nhận. Nếu cố truy cập port khác (port 22), in log cảnh báo trên terminal cho quản trị viên.

- Role = “IT”, có toàn quyền truy cập và không giới hạn thời gian đến app_server cụ thể có thể đọc (port 80) và thêm, xóa, sửa (port 22).
- Các kết nối trong mạng nội bộ (trừ app_server cần xét theo policy) mặc định được cho phép.

b.3. Cơ chế cài đặt flow rule

Sau khi đối chiếu yêu cầu với các policy, controller tiến hành cài đặt flow rule (thêm hoặc loại bỏ) xuống các switch.

```
if allowed:
    self.add_flow(dp, 10, match, actions)
    self.logger.info("ALLOW %s(%s) → %s:%s | %s",
                     ip_pkt.src, self._get_role(ip_pkt.src),
                     ip_pkt.dst, dst_port, reason)
else:
    self.add_drop(dp, match)
    self.logger.warning("DENY %s(%s) → %s:%s | %s",
                        ip_pkt.src, self._get_role(ip_pkt.src),
                        ip_pkt.dst, dst_port, reason)
    return
```

Hình 5.20. Cấu hình controller cho kịch bản 2 (6)

b.4. Quản lý flow rule time out

Ở đây, nhóm cài đặt idle timeout là 120s, trong thời gian này, flow rule sẽ được giữ lại và áp dụng cho các yêu cầu mới nếu có cùng thông tin thiết bị và loại yêu cầu. Sau thời gian này, flow rule sẽ tự động bị loại bỏ, các yêu cầu mới sẽ được gửi tới controller để xem xét có được thông qua hay không.

```
def add_flow(self, dp, priority, match, actions, idle=120):
    parser = dp.ofproto_parser
    ofp = dp.ofproto
    inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS, actions)]
    dp.send_msg(parser.OFPFlowMod(
        datapath=dp,
        priority=priority,
        match=match,
        instructions=inst,
        idle_timeout=idle
    ))

def add_drop(self, dp, match, idle=120):
    parser = dp.ofproto_parser
    dp.send_msg(parser.OFPFlowMod(
        datapath=dp,
        priority=200,
        match=match,
        instructions=[],
        idle_timeout=idle
    ))
```

Hình 5.21. Cấu hình controller cho kịch bản 2 (7)

c. Demo

- Khởi tạo một topology bằng mininet, kết nối OpenVSwitch.

```python
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import RemoteController
from mininet.cli import CLI
from mininet.log import setLogLevel
from mininet.node import OVSSwitch

class MyTopo( Topo ):
    "Simple topology example."

    def build( self ):
        #Add router
        router = self.addHost('r')

        # Add hosts
        Host01 = self.addHost( 'sales1', ip='10.0.1.1/24', defaultRoute='via 10.0.1.254')
        Host02 = self.addHost( 'sales2', ip='10.0.1.2/24', defaultRoute='via 10.0.1.254' )
        Host03 = self.addHost( 'sales3', ip='10.0.1.3/24', defaultRoute='via 10.0.1.254' )
        Host04 = self.addHost( 'sales4', ip='10.0.1.4/24', defaultRoute='via 10.0.1.254' )

        Host05 = self.addHost( 'it1', ip='10.0.2.1/24', defaultRoute='via 10.0.2.254' )
        Host06 = self.addHost( 'it2', ip='10.0.2.2/24', defaultRoute='via 10.0.2.254' )
        Host07 = self.addHost( 'it3', ip='10.0.2.3/24', defaultRoute='via 10.0.2.254' )
        Host08 = self.addHost( 'it4', ip='10.0.2.4/24', defaultRoute='via 10.0.2.254' )

        Host09 = self.addHost( 'visitor1', ip='10.0.4.1/24', defaultRoute='via 10.0.4.254' )
        Host10 = self.addHost( 'visitor2', ip='10.0.4.2/24', defaultRoute='via 10.0.4.254' )
        Host11 = self.addHost( 'visitor3', ip='10.0.4.3/24', defaultRoute='via 10.0.4.254' )
        Host12 = self.addHost( 'visitor4', ip='10.0.4.4/24', defaultRoute='via 10.0.4.254' )

        app = self.addHost('app', ip='10.0.3.10/24', defaultRoute='via 10.0.3.254')


```

Hình 5.22. Cấu hình topology (1)

```python
def setup_network():
    net = Mininet(
        topo=MyTopo(),
        controller=None,
        autoSetMacs=True
    )

    c0 = net.addController(
        'c0',
        controller=RemoteController,
        ip='127.0.0.1',
        port=6653
    )

    net.start()

    # Configure Router
    r = net.get('r')
    r.cmd("ifconfig r-eth0 10.0.1.254/24")
    r.cmd("ifconfig r-eth1 10.0.2.254/24")
    r.cmd("ifconfig r-eth2 10.0.3.254/24")
    r.cmd("ifconfig r-eth3 10.0.4.254/24")
    r.cmd("sysctl -w net.ipv4.ip_forward=1")

    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    setup_network()

# For mn --custom
topos = { 'mytopo': (lambda: MyTopo()) }
```

Hình 5.23. Cấu hình topology (2)

```
thanh@ubuntu:~/Documents/rabac$ sudo python3 topo.py
*** Creating network
*** Adding hosts:
app it1 it2 it3 it4 r sales1 sales2 sales3 sales4 visitor1 visitor2 visitor3 visitor4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(app, s3) (it1, s2) (it2, s2) (it3, s2) (it4, s2) (r, s1) (r, s2) (r, s3) (r, s4) (sales1, s1) (sales2, s1) (sales3, s1) (sales4, s1) (visitor1, s4) (visitor2, s4) (visitor3, s4) (visitor4, s4)
*** Configuring hosts
app it1 it2 it3 it4 r sales1 sales2 sales3 sales4 visitor1 visitor2 visitor3 visitor4
Unable to contact the remote controller at 127.0.0.1:6653
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
```

Hình 5.24. Khởi chạy topology

- Khởi tạo Ryu controller.

```
thanh@ubuntu:~/Documents/rabac$ ryu-manager controller.py
loading app controller.py
loading app ryu.controller.ofp_handler
instantiating app controller.py of PolicyRBAC
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Hình 5.25. Khởi chạy controller

- Sử dụng xterm để truy cập app, bật dịch vụ http_server và ssh. Sau đó, tạo một file tài liệu chung có tên secret.txt với nội dung SECRET DATA.

```
mininet> xterm app
mininet>
mininet>
mininet>
mininet>
mininet>
mininet>
mininet>
mininet>
```

"Node: app"

```
root@ubuntu:/home/thanh/Documents/rabac# cat secret.txt
secret data
root@ubuntu:/home/thanh/Documents/rabac#
root@ubuntu:/home/thanh/Documents/rabac# /usr/sbin/sshd -D &
[1] 4994
root@ubuntu:/home/thanh/Documents/rabac# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

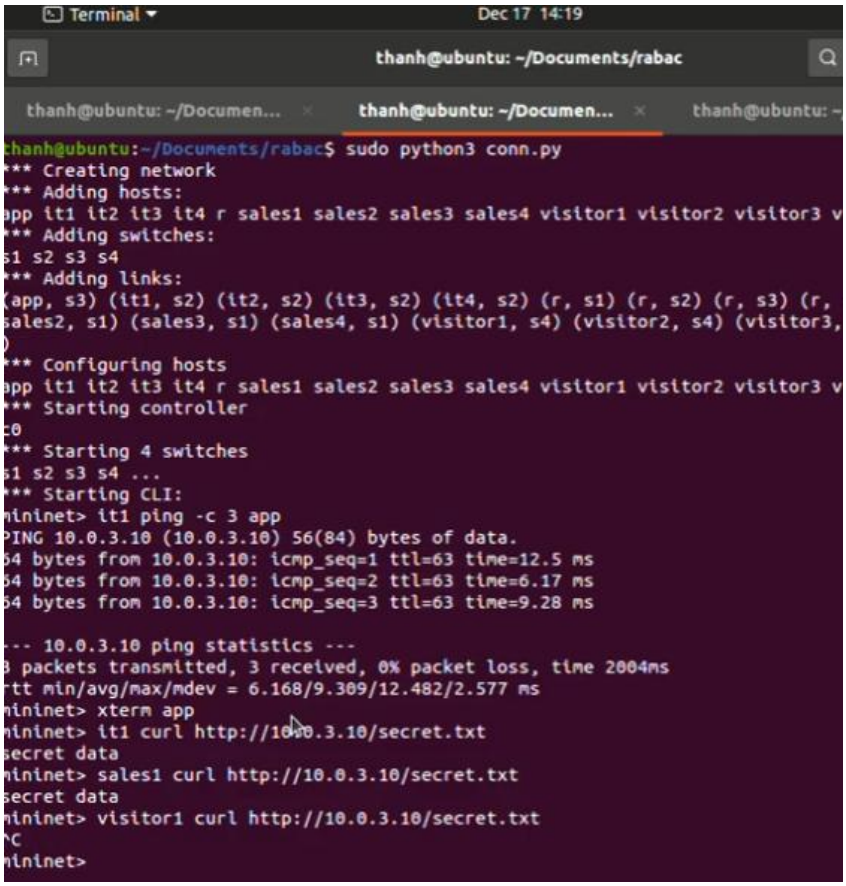
Hình 5.26. Bật dịch vụ http_server và ssh trên app_server

- Test thử ping trong mạng nội bộ, từ it1 đến app_server.

```
mininet> it1 ping -c 3 app
PING 10.0.3.10 (10.0.3.10) 56(84) bytes of data.
64 bytes from 10.0.3.10: icmp_seq=1 ttl=63 time=12.5 ms
64 bytes from 10.0.3.10: icmp_seq=2 ttl=63 time=6.17 ms
64 bytes from 10.0.3.10: icmp_seq=3 ttl=63 time=9.28 ms
```

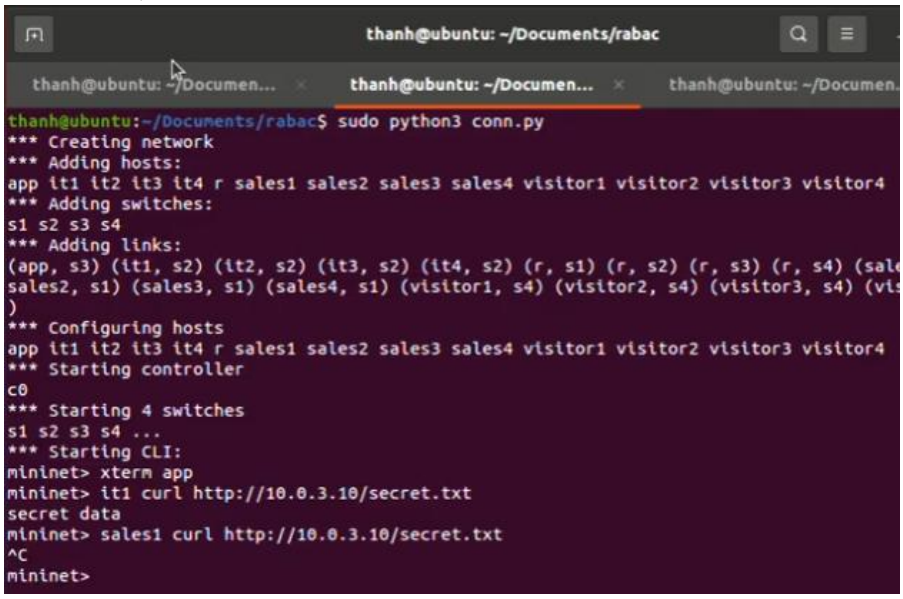
Hình 5.27. Test ping nội bộ

- Lần lượt thử từng role là IT (it1), Sales (sales1) và Visitor (visitor1) gửi yêu cầu đọc đến app_server. Kết quả cho thấy, chỉ có yêu cầu từ thiết bị it1 và sales1 là được chấp nhận, vì thời gian hiện tại trong test case đang là 14h19 phù hợp với thời gian làm việc nên yêu cầu của sales1 được chấp nhận.

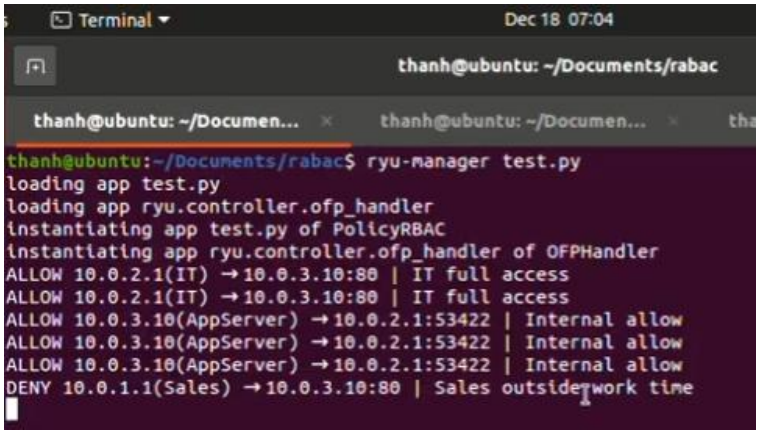


Hình 5.28. Kết quả đọc nội dung tài liệu chung (trong giờ làm việc)

- Trường hợp nằm ngoài thời gian làm việc, cụ thể ở hình dưới là 7h01, yêu cầu của sales1 sẽ bị từ chối, và controller sẽ in ra cảnh báo trên teminal.

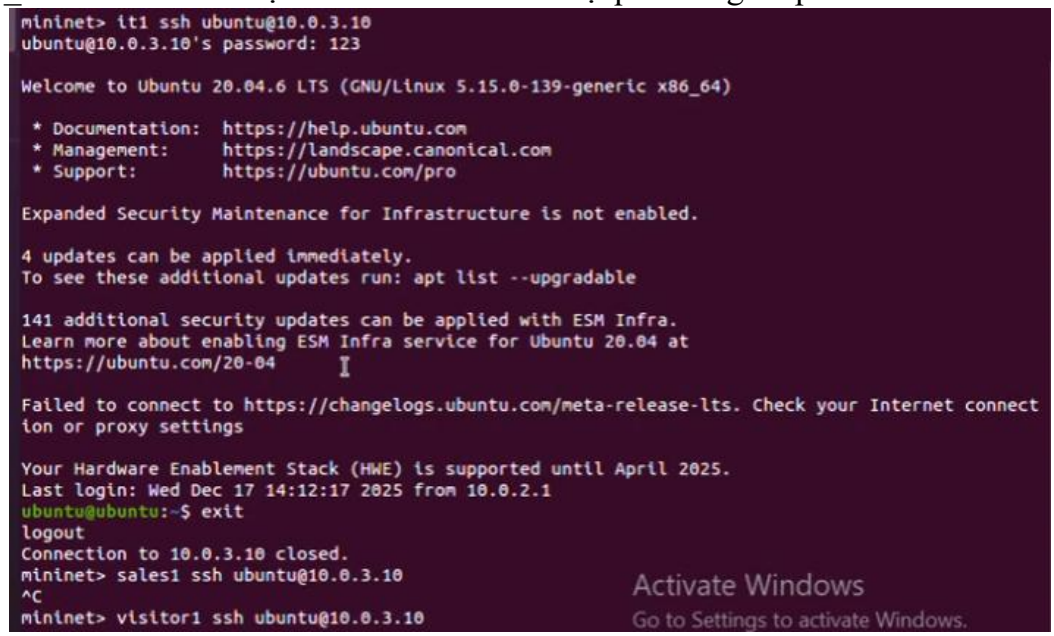


Hình 5.29. Kết quả đọc nội dung tài liệu chung (ngoài giờ làm việc)



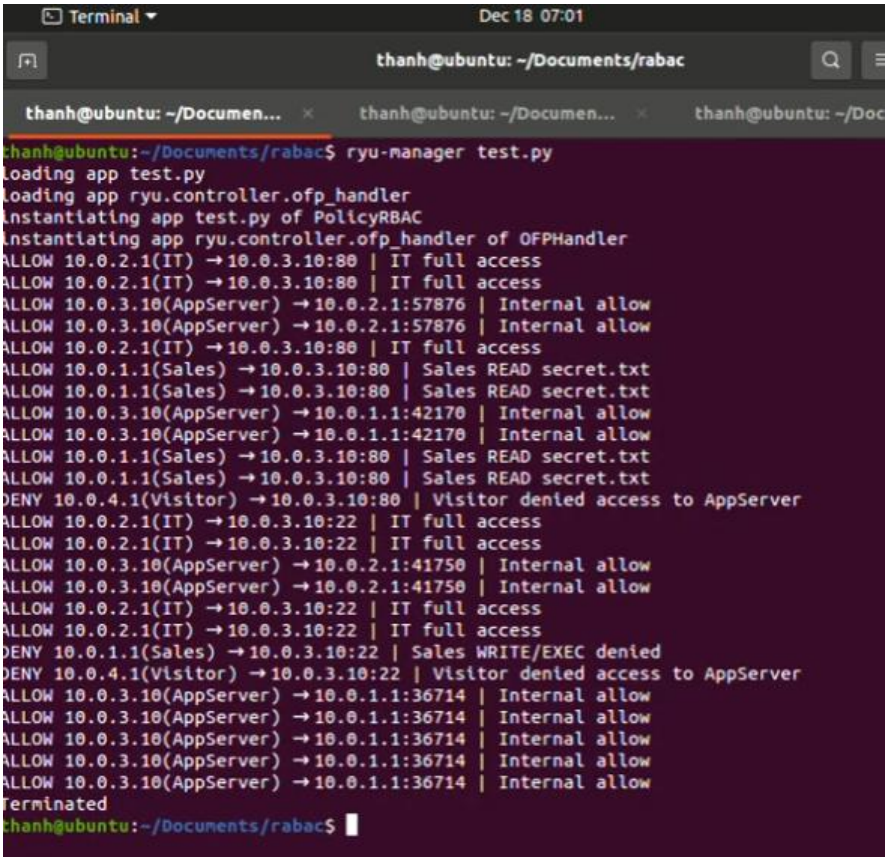
Hình 5.30. Log phía controller (ngoài giờ làm việc)

- Tương tự, thử nghiệm từng role gửi yêu cầu đến dịch vụ ssh ở phía app_server, như các cài đặt policy ở controller, chỉ có thiết bị it1 là thực hiện ssh thành công đến app_server. Hai thiết bị là sales1 và visitor1 bị quá thời gian phản hồi.



Hình 5.31. Kết quả ssh đến app_server

- Log trên terminal của controller được in ra sau tất cả các testcase trên (trừ testcase sales truy cập ngoài thời gian làm việc).



Hình 5.32. Log phía controller (trong giờ làm việc)

Link video: https://www.youtube.com/watch?v=_dxlEk1mQis

- Các test case trong demo
- Test 1: IT request tài liệu đến AppServer.
- Test 2: Sales request tài liệu đến AppServer (trong thời gian làm việc).
- Test 3: Visitor request tài liệu đến AppServer.
- Test 4: IT ssh đến AppServer.
- Test 5: Sales ssh đến AppServer.
- Test 6: Visitor ssh đến AppServer.
- Test 7: Sales request tài liệu đến AppServer (ngoài thời gian làm việc).

6. Ưu nhược điểm

6.1. Kịch bản 1

- Ưu điểm
  - Dễ dàng mở rộng: vai trò của người dùng được xác định dựa trên subnet của địa chỉ IP, điều này giúp khi có sự thay đổi về số lượng nhân viên chỉ cần cấp

mới địa chỉ IP theo subnet mà không cần cập nhật chính sách mới ở Controller
    - Tốc độ quyết định nhanh chóng: quá trình so khớp giữa IP và chính sách được cài đặt diễn ra nhanh chóng, đảm bảo tiết kiệm tài nguyên và lưu lượng mạng ổn định.
    - Tự động cảnh báo: có giao diện thống kê và các log alert khi sự cố xảy ra.
    - Quản lí dễ dàng: quản trị viên chỉ cần điều chỉnh cấu hình DHCP hoặc VLAN để thay đổi Role của một thiết bị mà không cần tương tác trực tiếp với Controller.
- Nhược điểm
    - Phụ thuộc vào cấu trúc mạng vật lí: yêu cầu cấu trúc mạng vật lí phải được phân chia logic theo phòng ban hoặc vai trò.
    - Phụ thuộc vào controller: nếu controller bị lỗi, hệ thống sẽ không thể cập nhật rule.
    - Khả năng kiểm soát còn hạn chế: thiếu các yếu tố như thời gian làm việc, thiết bị, thuộc tính của tài nguyên.

6.2. Kịch bản 2

- Ưu điểm
    - Khả năng kiểm soát cao: có thể áp dụng các hính sách phân quyền có điều kiện, quyết định dựa trên nhiều thuộc tính vai trò, thời gian, tài nguyên và hành động. Từ đó tăng độ chính xác.
    - Nâng cao tính an toàn và bảo mật: giảm thiểu các rủi ro bị tấn công như việc tài khoản bị lạm đụng dể truy cập trái phép tài nguyên công ty ngoài giờ làm việc
    - Tự động hóa cao: khi role hoặc time thay đổi, controller sẽ tự cập nhật flow rule tương ứng.
    - Hỗ trợ phát hiện các truy cập trái phép: có giao diện và log alert.
- Nhược điểm
    - Chính sách phức tạp: yêu cầu thu thập và phân tích dựa trên nhiều nhiều thuộc tính, chính sách khó viết, khó bảo trì.
    - Đồng bộ hóa thời gian: controller và các thiết bị liên quan cần có cơ chế đồng bộ hóa thời gian chính xác (NTP) để đảm bảo quyết định truy cập chính xác khi chuyển giao khung giờ.
    - Tốn tài nguyên: Nếu mạng lớn và có nhiều thuộc tính, việc đánh giá và phân tích sẽ tốn khá nhiều thời gian và CPU.
    - Triển khai phức tạp: cần đồng bộ role, time, resource classification, mô hình hoá tài nguyên rõ ràng

7. Tổng kết

Hệ thống Dynamic Access Control trong môi trường SDN mang lại khả năng kiểm soát truy cập linh hoạt, tự động và hiệu quả hơn so với các phương pháp truyền thống.

Thông qua cơ chế phân tích Packet-In và cài đặt flow rule theo nhu cầu, controller đảm bảo các quyết định truy cập luôn bám sát chính sách hiện tại. Việc gán vai trò động dựa trên subnet giúp tăng tính tự động hóa và khả năng mở rộng khi số lượng thiết bị tăng lên. Đồng thời, cơ chế timeout cho phép loại bỏ các flow rule lỗi thời, đảm bảo mạng luôn vận hành theo đúng policy mới nhất.

Trong khi đó, mô hình ABAC dựa trên thuộc tính giúp nâng cao mức độ bảo mật của doanh nghiệp bằng cách dựa trên các chính sách chi tiết và linh hoạt. Sự kết hợp giữa SDN và ABAC tạo ra một hệ thống kiểm soát truy cập thông minh, tự động, có khả năng phát hiện và ngăn chặn truy cập trái phép theo thời gian thực.

Kết quả mô phỏng và các test case cho thấy giải pháp hoạt động ổn định, đáp ứng tốt yêu cầu của môi trường doanh nghiệp hiện đại.

TÀI LIỆU THAM KHẢO

[1] J. U. Waziri and E. Omokhuale, "Software Defined Networking (SDN) Controller for Dynamic Network Traffic Isolation Through OpenFlow," *Iconic Research and Engineering Journals*, vol. 6, no. 1, pp. 556–575, 2022. [Online]. Available: https://www.irejournals.com/paper-details/1703674. Accessed: Nov. 24, 2025.

[2] T. Fernando, "Software Defined Networking (SDN) & OpenFlow," *LinkedIn*. [Online]. Available: https://www.linkedin.com/pulse/software-defined-networking-sdn-openflow-trinal-fernando. Accessed: Nov. 24, 2025.

[3] X. Yao, H. Liu, L. Ning, Y. Xiang, *Security and privacy issues of physical objects in the IoT: Challenges and opportunities*, Digital Communications and Networks, vol. 6, no. 4, pp. 445–456, 2021, doi: 10.1016/j.dcan.2020.09.001.

[4] "What Is Dynamic Access Control?," *Ping Identity Blog*. [Online]. Available: https://www.pingidentity.com/en/resources/blog/post/dynamic-access-control.html. Accessed: Nov. 24, 2025.

[5] "Dynamic Access Control," *Securden*. [Online]. Available: https://www.securden.com/educational/dynamic-access-control.html. Accessed: Nov. 24, 2025.

[6] "What Is Attribute-Based Access Control (ABAC)?," *Rapid7*. [Online]. Available: https://www.rapid7.com/fundamentals/attribute-based-access-control-abac/. Accessed: Nov. 24, 2025.

[7] "What Is Attribute-Based Access Control (ABAC)?," *Infisign Blog*. [Online]. Available: https://www.infisign.ai/blog/what-is-attribute-based-control-abac. Accessed: Nov. 24, 2025.

[8] "Ryu SDN Framework," *Ryu Official Website*. [Online]. Available: https://ryu-sdn.org/. Accessed: Nov. 24, 2025.

[9] "Ryu Controller," *Scribd*. [Online]. Available: https://www.scribd.com/document/250915607/Ryu-controller. Accessed: Nov. 24, 2025.

[10] "SDN Controllers: Ryu and OpenDaylight," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/sdn-controllers-ryu-and-odl/. Accessed: Nov. 24, 2025.

[11] "What Is Open vSwitch?," *Open vSwitch Documentation*. [Online]. Available: https://docs.openvswitch.org/en/latest/intro/what-is-ovs/. Accessed: Nov. 24, 2025.

[12] "Data Centre Networking: What Is OVS?," *Ubuntu Blog*. [Online]. Available: https://ubuntu.com/blog/data-centre-networking-what-is-ovs. Accessed: Nov. 24, 2025.

[13] “Tổng quan về SDN và Open vSwitch,” *Viblo*. [Online]. Available: https://viblo.asia/p/tong-quan-ve-sdn-va-openvswitch-m68Z0N865kG. Accessed: Nov. 24, 2025.

[14] “Using Open vSwitch,” *IBM Documentation*. [Online]. Available: https://www.ibm.com/docs/en/linux-on-systems?topic=choices-using-open-vswitch. Accessed: Nov. 24, 2025.

[15] “Mininet Documentation,” *GitHub Wiki*. [Online]. Available: https://github.com/mininet/mininet/wiki/Documentation. Accessed: Nov. 24, 2025.